

Fondamentaux des API avec Python

FastAPI · Pydantic · REST · OpenAPI · Swagger UI

Objectifs

À la fin de cette séance, vous serez capables de :

- comprendre ce qu'est une API ;
 - expliquer le fonctionnement d'une communication client / serveur ;
 - comprendre les principes fondamentaux d'une API REST ;
 - utiliser les principales méthodes HTTP ;
 - créer une API REST avec **FastAPI** ;
 - définir des routes et leurs paramètres ;
 - recevoir et retourner des données JSON ;
 - créer des modèles avec **Pydantic** ;
 - valider automatiquement les données reçues ;
 - définir le contrat de réponse d'une API ;
 - comprendre le rôle d'**OpenAPI** ;
 - utiliser **Swagger UI** pour tester une API ;
 - comprendre comment FastAPI génère automatiquement sa documentation.
-

1. Pourquoi avons-nous besoin d'APIs ?

Imaginez un jeu vidéo sur smartphone. L'application doit afficher le score et le niveau de la joueuse, Alice.

Ces informations sont stockées sur un serveur distant, dans une **base de données**.

graph TD

```
A["Application du jeu<br><br>Alice<br>Score : 1250<br>Niveau : 12"] -->|???| B["Base de
```

```
classDef client_data fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff
classDef result_db fill:#a5d6a7,stroke:#2e7d32,stroke-width:2px,color:#000
class A client_data
class B result_db
```

L'application de votre téléphone pourrait-elle **se connecter directement à cette base de données** pour lire et modifier les scores ?

Techniquement, oui. Mais ce serait une très mauvaise idée.

Si l'application communique directement avec la base de données, cela signifie que le téléphone contient les codes d'accès à cette base. Un joueur un peu bricoleur pourrait intercepter la connexion et dire à la base de données : *“Modifie mon score et mets 999 999 points”*.

De plus, l'application devrait calculer elle-même si le joueur a le droit de passer au niveau supérieur, ce qui rend les mises à jour complexes si l'on sort le jeu sur d'autres supports.

C'est pour cela que l'on intercale un **arbitre** : l'API.

```
graph TD
    A[Application du jeu] -->|Requête HTTP<br>J'ai battu un monstre| B[API<br>L'Arbitre]
    B -->|Vérifie les règles<br>Met à jour les points| C[(Base de données)]
    C -. ->|Nouveau score : 1260| B
    B -. ->|Affiche 1260| A

    classDef client fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff
    classDef api fill:#ffcc80,stroke:#e65100,stroke-width:2px,color:#000
    classDef db fill:#a5d6a7,stroke:#2e7d32,stroke-width:2px,color:#000

    class A client
    class B api
    class C db
```

L'application ne parle plus à la base de données. Elle envoie une requête à l'API : *“Alice vient de battre un monstre”*.

L'API, qui contient toutes les règles du jeu, vérifie que l'action est valide, calcule les points gagnés, met elle-même la base de données à jour en toute sécurité, puis renvoie le nouveau score au téléphone.

Cette couche intermédiaire est une **API**.

2. Qu'est-ce qu'une API ?

API signifie :

Application Programming Interface

Une API est une **interface permettant à un programme d'interagir avec un autre programme ou service**.

L'API définit notamment :

- comment demander une information ;
 - quelles informations envoyer ;
 - comment les données sont structurées ;
 - quelles réponses sont possibles ;
 - quelles erreurs peuvent être retournées.
-

Une API comme contrat

On peut voir une API comme un contrat entre deux applications.

```
flowchart TD
    C1["CLIENT<br>« Je voudrais le joueur 42 »"] -->|Requête| A["API (SERVEUR)"]
    A -->|Réponse| C2["CLIENT<br>« Voici le joueur 42 »"]

    %% Styles
    classDef client_data fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff
    classDef process_api fill:#ffcc80,stroke:#e65100,stroke-width:2px,color:#000

    class C1,C2 client_data
    class A process_api
```

Le client n'a pas besoin de connaître :

- le langage utilisé par le serveur ;
- la base de données ;
- l'architecture interne ;
- le code du serveur.

Il doit simplement connaître **le contrat de l'API**.

3. Exemple : utiliser une API existante

Python peut communiquer avec une API grâce à HTTP.

Par exemple :

```
import requests

response = requests.get(
    "https://api.github.com/users/python"
)

print(response.status_code)
print(response.json())
```

La fonction `requests.get()` envoie une requête HTTP.

Le serveur retourne une réponse.

Python

GET /users/python

GitHub API

JSON

Python

flowchart TD

P1[Python] -->|GET /users/python| G[GitHub API]

G -->|JSON| P2[Python]

%% Styles

classDef client_data fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff

classDef process_api fill:#ffcc80,stroke:#e65100,stroke-width:2px,color:#000

class P1,P2 client_data

class G process_api

Que contient une réponse HTTP ?

Une réponse possède notamment :

- un **code de statut** ;
- des **en-têtes** ;
- un **contenu**.

Exemple :

```
HTTP/1.1 200 OK

Content-Type: application/json

{
  "login": "python",
  "id": 1525981
}
```

4. Les codes HTTP

Quelques codes sont particulièrement importants.

Code	Signification
200	La requête a réussi
201	Une ressource a été créée
204	Succès, sans contenu à retourner
400	Requête incorrecte
401	Authentification nécessaire
403	Accès interdit
404	Ressource inexistante
422	Données reçues invalides
500	Erreur interne du serveur

Une API ne doit donc pas seulement retourner des données.

Elle doit également indiquer **ce qui s'est passé**.

5. Le format JSON

Les APIs REST utilisent très souvent le format **JSON**.

JSON permet de représenter des données structurées.

```
{
  "id": 42,
  "name": "Alice",
  "score": 1250
}
```

Un objet JSON peut contenir :

- des chaînes de caractères ;
- des nombres ;
- des booléens ;
- des tableaux ;
- d'autres objets.

Par exemple :

```
{
  "id": 42,
  "name": "Alice",
  "score": 1250,
  "weapons": [
    "sword",
    "bow"
  ],
  "active": true
}
```

6. API REST

REST signifie :

Representational State Transfer

REST est un ensemble de principes permettant notamment d'organiser une API autour de **ressources**.

Dans notre jeu, les ressources pourraient être :

```
players
games
weapons
scores
```

On peut alors imaginer :

```
GET    /players
GET    /players/42

POST   /players

PUT    /players/42

DELETE /players/42
```

7. Les méthodes HTTP

Les quatre méthodes principales que nous utiliserons sont :

Méthode	Action
GET	Lire
POST	Créer
PUT	Modifier
DELETE	Supprimer

On peut les rapprocher des opérations CRUD :

HTTP	CRUD
GET	Read
POST	Create
PUT	Update

HTTP	CRUD
DELETE	Delete

8. Notre fil rouge : Game API

Nous allons construire progressivement une API pour un petit jeu.

Notre API devra gérer des joueurs.

Un joueur possède :

```
id
name
score
level
```

Notre API devra permettre de :

Récupérer tous les joueurs

```
GET /players
```

Récupérer un joueur

```
GET /players/{player_id}
```

Créer un joueur

```
POST /players
```

Modifier un joueur

```
PUT /players/{player_id}
```

Supprimer un joueur

```
DELETE /players/{player_id}
```

9. Pourquoi FastAPI ?

Nous pourrions construire nous-mêmes un serveur HTTP en Python.

Mais il faudrait gérer beaucoup de choses :

- réception des requêtes ;
- routage ;
- parsing du JSON ;
- validation ;
- gestion des erreurs ;
- sérialisation ;
- documentation ;
- etc.

C'est précisément le rôle d'un framework.

Pour notre API, nous utiliserons :

FastAPI

FastAPI est un framework Python permettant de construire des APIs modernes.

Il s'appuie fortement sur :

- les **Type Hints Python** ;
- **Pydantic** ;
- un serveur **ASGI** ;
- **OpenAPI**.

10. Installer FastAPI

Créons un environnement virtuel :

```
python -m venv .venv
```

Activation sous Linux / macOS :

```
source .venv/bin/activate
```

Sous Windows :

```
.venv\Scripts\activate
```

Puis :

```
pip install "fastapi[standard]"
```

11. Notre première API

Créons un fichier :

```
main.py
```

Avec seulement quelques lignes :

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to the Game API"}
```

12. Lancer le serveur

Depuis le terminal :

```
fastapi dev main.py
```

Le serveur démarre localement.

Par défaut :

```
http://127.0.0.1:8000
```

Ouvrons cette adresse dans un navigateur.

Nous obtenons :

```
{  
  "message": "Welcome to the Game API"  
}
```

13. Que s'est-il passé ?

Lorsque nous avons écrit :

```
@app.get("/")
```

nous avons indiqué à FastAPI :

Lorsqu'une requête GET est envoyée vers /, exécute cette fonction.

```
GET /
```

```
FastAPI
```

```
home()
```

```
{"message": "..."}  
}
```

```

flowchart TD
    A["GET /"] --> B{FastAPI}
    B --> C["home()"]
    C --> D["{ 'message': '...' }"]

    %% Styles
    classDef client_data fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff
    classDef process_api fill:#ffcc80,stroke:#e65100,stroke-width:2px,color:#000
    classDef result_db fill:#a5d6a7,stroke:#2e7d32,stroke-width:2px,color:#000

    class A client_data
    class B,C process_api
    class D result_db

```

Cette association entre une URL et une fonction est appelée une **route**.

14. Les routes

Une route est définie par :

```

@app.get("/players")
def get_players():
    ...

```

On peut lire cela comme :

Pour une requête GET vers `/players`, exécuter `get_players()`.

Exemple :

```

@app.get("/players")
def get_players():
    return [
        {"id": 1, "name": "Alice", "score": 1200},
        {"id": 2, "name": "Bob", "score": 950},
    ]

```

Tester :

```
GET /players
```

Résultat :

```
[
  {
    "id": 1,
    "name": "Alice",
    "score": 1200
  },
  {
    "id": 2,
    "name": "Bob",
    "score": 950
  }
]
```

15. Paramètres de chemin

Comment récupérer uniquement le joueur 42 ?

Nous pouvons utiliser un paramètre dans l'URL :

```
@app.get("/players/{player_id}")
def get_player(player_id):
    return {
        "id": player_id
    }
```

Une requête :

```
GET /players/42
```

donne :

```
{
  "id": "42"
}
```

16. Les Type Hints

Nous voulons que `player_id` soit un entier.

Nous écrivons :

```
@app.get("/players/{player_id}")
def get_player(player_id: int):
    return {
        "id": player_id
    }
```

Maintenant :

```
/players/42
```

est valide.

Mais :

```
/players/hello
```

ne l'est pas.

FastAPI connaît le type attendu grâce au **Type Hint** :

```
player_id: int
```

17. Les Type Hints deviennent un outil de validation

C'est une caractéristique importante de FastAPI.

Dans Python :

```
player_id: int
```

est normalement une annotation de type.

FastAPI utilise cette information pour :

- analyser la requête ;
- convertir les données ;
- valider les données ;
- générer la documentation.

Type Hint

FastAPI

Validation Docs Conversion

```

flowchart TD
    A[Type Hint] --> B[FastAPI]
    B --> C[Validation]
    B --> D[Docs]
    B --> E[Conversion]

    %% Styles
    classDef client_data fill:#2B5E83,stroke:#fff,stroke-width:2px,color:#fff
    classDef process_api fill:#ffcc80,stroke:#e65100,stroke-width:2px,color:#000
    classDef result_db fill:#a5d6a7,stroke:#2e7d32,stroke-width:2px,color:#000

    class A client_data
    class B process_api
    class C,D,E result_db
  
```

18. Où sont les joueurs ?

Pour le moment, nous allons utiliser une simple liste Python.

```

players = [
    {
        "id": 1,
  
```

```
[
    {
        "name": "Alice",
        "score": 1200,
        "level": 12,
    },
    {
        "id": 2,
        "name": "Bob",
        "score": 950,
        "level": 9,
    },
]
```

Notre route :

```
@app.get("/players")
def get_players():
    return players
```

19. Récupérer un joueur

Nous pouvons rechercher le joueur dans notre liste :

```
@app.get("/players/{player_id}")
def get_player(player_id: int):
    for player in players:
        if player["id"] == player_id:
            return player

    return {"error": "Player not found"}
```

Nous pouvons maintenant appeler :

```
GET /players/1
```

ou :


```
GET /players/2
```

20. Un problème apparaît...

Notre API doit également permettre de **créer** un joueur.

Nous voulons recevoir :

```
{  
    "name": "Charlie",  
    "score": 1000,  
    "level": 1  
}
```

Mais comment indiquer à FastAPI :

- quels champs sont obligatoires ?
- quels sont leurs types ?
- quelles valeurs sont acceptées ?
- à quoi doit ressembler le JSON ?

C'est ici que **Pydantic** intervient.

21. Pydantic

Pydantic permet de définir des modèles de données Python.

Nous créons une classe :

```
from pydantic import BaseModel  
  
class Player(BaseModel):  
    name: str  
    score: int  
    level: int
```

Nous venons de définir le modèle d'un joueur.

22. Le modèle comme contrat

Notre modèle signifie :

Player

```
name  → string
score → integer
level → integer
```

On peut donc considérer **Player** comme un **contrat de données**.

JSON reçu

```
Pydantic
Player
```

Données validées

23. Utiliser un modèle dans une route

Nous pouvons maintenant écrire :

```
@app.post("/players")
def create_player(player: Player):
    return player
```

Le Type Hint :

```
player: Player
```

indique à FastAPI :

Le corps de la requête doit respecter le modèle **Player**.

24. Tester avec JSON

Une requête POST peut contenir :

```
{  
  "name": "Charlie",  
  "score": 1000,  
  "level": 1  
}
```

FastAPI :

1. reçoit le JSON ;
2. demande à Pydantic de le valider ;
3. construit un objet **Player** ;
4. appelle notre fonction.

JSON

FastAPI

Pydantic

valide

Player

create_player()

25. Et si les données sont incorrectes ?

Essayons :

```
{  
    "name": "Charlie",  
    "score": "hello",  
    "level": 1  
}
```

Le champ `score` doit être un entier.

FastAPI détecte automatiquement le problème.

Nous n'avons pas écrit :

```
if not isinstance(score, int):  
    ...
```

La validation est réalisée automatiquement.

26. Ajouter des contraintes

Les types ne sont pas toujours suffisants.

Par exemple :

Le score ne peut pas être négatif.

Nous pouvons utiliser `Field`.

```
from pydantic import BaseModel, Field  
  
class Player(BaseModel):  
    name: str = Field(min_length=3)  
    score: int = Field(ge=0)  
    level: int = Field(ge=1)
```

Nous exprimons maintenant des règles métier simples :

```
name → au moins 3 caractères  
score → >= 0  
level → >= 1
```

27. Pourquoi est-ce intéressant ?

Nous avons défini les règles **une seule fois**.

```
class Player(BaseModel):  
    name: str = Field(min_length=3)  
    score: int = Field(ge=0)  
    level: int = Field(ge=1)
```

FastAPI/Pydantic peuvent ensuite utiliser ces informations pour :

- valider les données ;
- générer des erreurs ;
- construire le schéma OpenAPI ;
- documenter automatiquement l'API.

Player

Validation OpenAPI Documentation

28. Repenser notre API

Nous avons maintenant :

```
GET    /players
GET    /players/{player_id}

POST   /players

PUT    /players/{player_id}

DELETE /players/{player_id}
```

Nous avons donc les principaux éléments d'une API REST.



29. Le contrat d'une API

Une API doit permettre au client de savoir :

« Comment dois-je communiquer avec le serveur ? »

Par exemple :

```
POST /players
```

Request body:

```
{
  "name": "Charlie",
  "score": 1000,
  "level": 1
}
```

Le client doit savoir :

- que la route existe ;
- qu'elle utilise `POST` ;
- quels champs envoyer ;
- quels sont leurs types ;
- quels champs sont obligatoires ;
- quelle réponse attendre ;
- quelles erreurs peuvent être retournées.

Tout cela constitue une partie du **contrat de l'API**.

30. Le problème de la documentation

Imaginez une API avec :

```
50 endpoints
```

Comment documenter tout cela ?

On pourrait écrire un document à la main :

```
GET /players
```

```
Description :
```

```
Récupère la liste des joueurs.
```

```
Paramètres :
```

```
...
```

```
Réponse :
```

```
...
```

Mais cela présente un problème :

Le code et la documentation peuvent finir par ne plus correspondre.

FastAPI propose une autre approche.

31. OpenAPI

OpenAPI est une spécification permettant de décrire une API.

Elle permet notamment de décrire :

- les routes ;
- les méthodes HTTP ;
- les paramètres ;
- les modèles ;
- les données d'entrée ;
- les données de sortie ;
- les réponses possibles.

FastAPI génère automatiquement cette description.

32. OpenAPI JSON

Notre serveur expose automatiquement :

```
/openapi.json
```

Par exemple :

```
http://127.0.0.1:8000/openapi.json
```

Nous obtenons un document JSON décrivant notre API.

Il contient notamment :

```
paths
components
schemas
parameters
responses
...
```

Ce document constitue le **contrat machine-readable** de notre API.

33. De FastAPI à OpenAPI

Voici le mécanisme fondamental à retenir :



34. Swagger UI

FastAPI génère automatiquement une interface interactive :

`/docs`

Par exemple :

`http://127.0.0.1:8000/docs`

Cette interface est appelée :

Swagger UI

Elle permet de :

- voir les endpoints ;
 - voir les paramètres ;
 - voir les modèles ;
 - tester les requêtes ;
 - observer les réponses.
-

35. Tester une API avec Swagger

Dans Swagger UI :

```
GET /players
```

Puis :

Try it out

et :

Execute

Swagger envoie réellement la requête à notre serveur.

Nous obtenons par exemple :

Request URL:

```
http://127.0.0.1:8000/players
```

Response code:

```
200
```

Puis :

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "score": 1200,  
    "level": 12  
  }  
]
```

36. Swagger n'est pas l'API

Attention à une distinction importante.



OpenAPI décrit l'API.

Swagger UI est une interface permettant de visualiser et tester cette description.

37. ReDoc

FastAPI fournit également une autre interface :

`/redoc`

Par exemple :

`http://127.0.0.1:8000/redoc`

Nous avons donc :

URL	Rôle
<code>/docs</code>	Swagger UI
<code>/redoc</code>	ReDoc
<code>/openapi.json</code>	Contrat OpenAPI

38. Pourquoi cette génération automatique est intéressante ?

Nous avons écrit :

```
class Player(BaseModel):  
    name: str = Field(min_length=3)  
    score: int = Field(ge=0)  
    level: int = Field(ge=1)
```

Et :

```
@app.post("/players")  
def create_player(player: Player):  
    return player
```

FastAPI peut en déduire :

```
POST /players  
  
Body:  
Player  
  
name → string, minimum 3 caractères  
score → integer, minimum 0  
level → integer, minimum 1
```

Nous n'avons pas écrit cette documentation manuellement.

39. Le rôle des Type Hints

C'est probablement l'une des idées les plus importantes de FastAPI.

Dans FastAPI, les Type Hints ne servent pas uniquement à rendre le code plus lisible.

Ils permettent au framework de comprendre notre API.

Exemple :

```
@app.get("/players/{player_id}")
def get_player(player_id: int):
    ...
```

FastAPI comprend :

```
player_id

integer
```

Et :

```
def create_player(player: Player):
```

lui indique :

```
request body

Player
```

40. Entrées et sorties

Une API possède deux directions de données.



Pour les données reçues :

```
def create_player(player: Player):
```

Pour les données retournées :

```
def get_player(...) -> Player:
```

ou explicitement :

```
@app.get(
    "/players/{player_id}",
    response_model=Player
)
def get_player(player_id: int):
    ...
```

41. Pourquoi définir un modèle de réponse ?

Le modèle de réponse permet de définir ce que l'API doit retourner.

Par exemple :

```
class Player(BaseModel):
    id: int
    name: str
    score: int
    level: int
```

Puis :

```
@app.get("/players/{player_id}", response_model=Player)
def get_player(player_id: int):
    ...
```

Nous avons maintenant un contrat explicite :

```
GET /players/{player_id}
```

Response:

```
{
  "id": integer,
  "name": string,
  "score": integer,
  "level": integer
}
```

42. Un modèle pour la création, un autre pour la réponse

Dans une véritable application, nous ne voulons pas forcément utiliser le même modèle partout.

Par exemple :

```
class PlayerCreate(BaseModel):
    name: str
    score: int
    level: int
```

Et :

```
class Player(BaseModel):
    id: int
    name: str
    score: int
    level: int
```

Pourquoi ?

Parce que l'id est généré par le serveur.

Le client ne doit donc pas nécessairement l'envoyer.

Client	Server
name	
score	
level	
	id
	généré par le serveur

43. Notre API complète

Nous pouvons maintenant imaginer notre première version :

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class PlayerCreate(BaseModel):
    name: str = Field(min_length=3)
    score: int = Field(ge=0)
    level: int = Field(ge=1)

class Player(PlayerCreate):
    id: int

@app.get("/players")
def get_players():
    ...
```



```

@app.get("/players/{player_id}")
def get_player(player_id: int):
    ...

@app.post("/players")
def create_player(player: PlayerCreate):
    ...

@app.put("/players/{player_id}")
def update_player(
    player_id: int,
    player: PlayerCreate,
):
    ...

@app.delete("/players/{player_id}")
def delete_player(player_id: int):
    ...

```

Nous avons déjà la structure d'une véritable API REST.

44. Architecture conceptuelle

À ce stade, il faut avoir compris cette chaîne :



Code Python

Données

JSON

Et parallèlement :

FastAPI

Type Hints

Pydantic

OpenAPI

Swagger UI
/docs

ReDoc
/redoc

45. Défi — concevoir une API

À vous de jouer

Notre jeu possède maintenant des **armes**.

Une arme possède :

```
id
name
damage
weight
```

Proposez les endpoints nécessaires pour :

1. récupérer toutes les armes ;
2. récupérer une arme ;
3. créer une arme ;
4. modifier une arme ;
5. supprimer une arme.

Essayez de respecter les conventions REST.

46. Correction

Une solution possible :

```
GET    /weapons
GET    /weapons/{weapon_id}

POST   /weapons

PUT    /weapons/{weapon_id}

DELETE /weapons/{weapon_id}
```

On retrouve :

```
GET    → lecture
POST   → création
PUT    → modification
DELETE → suppression
```

47. Défi — définir le modèle Pydantic

Nous voulons qu'une arme respecte les règles suivantes :

```
name    → au moins 3 caractères  
damage  → entier positif  
weight  → nombre positif
```

Comment définir le modèle ?

48. Correction

```
from pydantic import BaseModel, Field  
  
class WeaponCreate(BaseModel):  
    name: str = Field(min_length=3)  
    damage: int = Field(gt=0)  
    weight: float = Field(gt=0)
```

FastAPI peut maintenant utiliser ce modèle pour :

- valider les requêtes ;
 - générer le schéma OpenAPI ;
 - documenter automatiquement les champs.
-

49. Défi — trouver l'information

Notre API est lancée.

Vous savez qu'elle possède :

```
GET /players  
POST /players  
GET /weapons  
POST /weapons
```

Mais vous ne connaissez pas tous les détails.

Question

Comment découvrir automatiquement comment utiliser cette API ?

50. Réponse

Avec Swagger UI :

```
/docs
```

ou avec ReDoc :

```
/redoc
```

Et pour obtenir directement le contrat machine-readable :

```
/openapi.json
```

51. Ce qu'il faut retenir

API

Une API permet à des applications de communiquer entre elles.

```
Client ← HTTP → Serveur
```

REST

Une API REST organise généralement les opérations autour de ressources :

```
GET
POST
PUT
DELETE
```

FastAPI

FastAPI permet de créer rapidement des APIs Python.

```
@app.get("/players")
def get_players():
    ...
```

Pydantic

Pydantic permet de définir et valider les données :

```
class Player(BaseModel):
    name: str
    score: int
```

Type Hints

FastAPI exploite les annotations de type :

```
player_id: int
```

OpenAPI

OpenAPI décrit automatiquement notre API.

```
/openapi.json
```

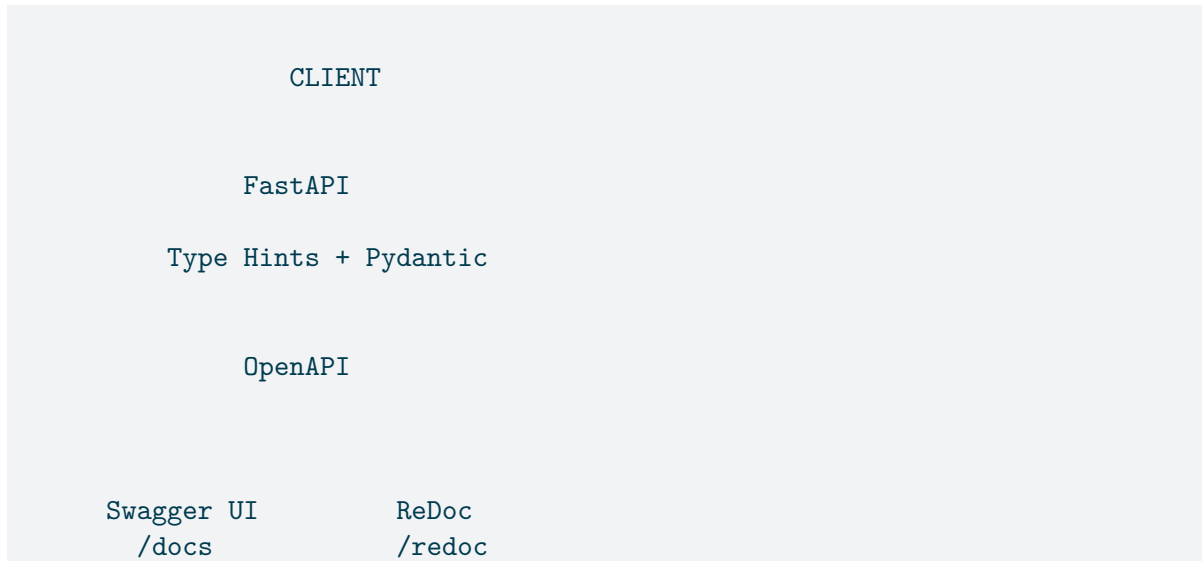
Swagger UI

Swagger UI fournit une interface interactive :

```
/docs
```

52. Le schéma à retenir





53. Pour le projet fil rouge

Pour la première partie de votre projet, vous devrez être capables de :

1. Créer une application FastAPI

```
from fastapi import FastAPI

app = FastAPI()
```

2. Définir des routes

```
@app.get("/")
@app.post("/")
@app.put("/")
@app.delete("/")
```


3. Utiliser les Type Hints

```
player_id: int
```

4. Créer des modèles Pydantic

```
class Player(BaseModel):  
    ...
```

5. Valider les données

```
score: int = Field(ge=0)
```

6. Définir les réponses

```
response_model=Player
```

7. Vérifier la documentation

```
/docs  
/redoc  
/openapi.json
```

54. La philosophie FastAPI

Une dernière idée à retenir :

Décrire correctement son API dans le code permet à FastAPI de faire beaucoup de travail automatiquement.

Code Python

+

Type Hints

+

Pydantic

FastAPI

Validation

Sérialisation

Gestion des erreurs

OpenAPI

Swagger UI